

Centre d'aide — un agent qui lit la documentation et le compte de l'utilisateur

Module Engagement — Zennyt Plan d'instruction et de réalisation. État au 15 août 2026.

Ce que ce document établit

Le centre d'aide est à construire autour d'un **agent**, et non d'un simple robot de conversation. L'intuition de départ est la bonne, et la maquette la confirme : quand l'utilisateur écrit « je n'accède pas à ma liste de candidats », la réponse attendue est « j'ai vérifié votre compte » — aucune base documentaire ne peut produire cette phrase.

Il faut donc **deux capacités distinctes**, et c'est le cœur de ce plan :

Question de l'utilisateur	Ce qu'il faut pour y répondre
« Comment est calculé mon Fit Score ? »	De la documentation — la réponse est la même pour tout le monde
« Pourquoi mon Fit Score est-il vide sur cette offre ? »	Son compte à lui — la réponse dépend de ses données

La première relève d'un RAG. La seconde relève d'**outils de lecture** que l'agent appelle. Confondre les deux est l'erreur qui produit un assistant qui invente.

1. Ce qui existe déjà — vérifié

L'interface est construite. Elle est arrivée avec la fusion d'IntergrationV1 le 15 août, et correspond à la maquette écran par écran.

Élément	Fichier	Lignes
Écran de discussion	help_chat_detail_page.dart	468
Dialogue Poor / OK / Great	rate_experience_dialog.dart	129
Formulaire « What can we improve? »	feedback_bottom_sheet.dart	126
Bulles de message	help_message_bubble.dart	105
Liste des conversations	help_center_page.dart	99
Animation « is typing... »	_TypingDots	—

Le socle serveur existe aussi : tables engagement.help_chats et engagement.help_messages, domaine HelpChat avec recordMessage(text, fromUser), et trois endpoints — lister les conversations, lire les messages, envoyer un message.

Ce qui manque n'est pas l'écran, c'est le comportement :

Manque	Constat
Personne ne répond	SendHelpMessageUseCase n'écrit que le message de l'utilisateur. La colonne from_user = false existe, rien ne l'écrit jamais .
On ne peut pas envoyer	Le datasource mobile n'expose que la lecture. La barre de saisie n'a qu'un micro et un « + » — aucun bouton d'envoi .
On ne peut pas ouvrir une conversation	Pas de POST /help-chats. La table contient 0 ligne .
La note part à la poubelle	_selectRating fait un setState, rien de plus.
Le commentaire aussi	onSubmit: (feedback) change l'affichage et jette le texte .

Manque	Constat
Le « is typing » est décoratif	Animation locale, déclenchée par rien de réel.

2. Trois découvertes qui cadrent le travail

2.1 Le RAG n'a pas de corpus — et le corpus disponible est dangereux

Il n'existe aucune documentation destinée aux utilisateurs. Pas une page.

Le dépôt contient bien 30 fichiers Markdown et **83 000 mots** — mais ce sont des documents **internes** : comptes rendus d'avancement, audits de défauts, plans d'intégration, discussions de clés d'API, noms des membres de l'équipe, décisions non annoncées.

Les indexer serait une fuite, pas une fonctionnalité. Un utilisateur qui demande « c'est quoi le Fit Score ? » recevrait une réponse nourrie d'un document expliquant que la pondération n'est pas calibrée, que tel calcul a été faux pendant deux jours, et que telle clé traîne dans un historique.

Conséquence sur l'ordre des travaux : le corpus est à écrire, et c'est le chemin critique. Ce n'est pas du code, et ça ne peut pas être sous-traité à l'agent lui-même.

2.2 Le « Wallet » de la maquette n'existe pas

La maquette montre un utilisateur qui se plaint de son *Wallet*. J'ai cherché : **aucune trace de portefeuille** dans le projet — ni backend, ni mobile, ni base.

La maquette est donc un gabarit générique, pas un scénario Zennyt. **Ne construisez pas l'agent autour de ses exemples** : les vraies questions porteront sur les candidatures, les tests, les jeux, le Fit Score et le profil.

2.3 L'infrastructure nécessaire est déjà là

Brique	État
Groq	Déjà utilisé pour la génération de tests et les résumés. API compatible OpenAI, donc l'appel d'outils est disponible .
Embeddings	EmbeddingPort + multilingual-e5-small. Aucune clé configurée — c'est NoOpEmbeddingPort qui répond aujourd'hui.
Stockage des empreintes	EmbeddingCodec : JSON dans une colonne text, similarité cosinus en mémoire. Documenté comme un choix assumé pour « un volume modeste ». Le même choix tient pour un corpus d'aide — quelques centaines de fragments. Pas besoin de pgvector.
Registre de ton	ResumeAudience distingue déjà CANDIDATE et RECRUITER, avec deux consignes de ton. Réutilisable tel quel.

3. L'architecture proposée

```

L'utilisateur écrit
|
v
+-----+
| L'agent (Groq) |
| 1. cherche dans la DOCUMENTATION | <- RAG, identique pour tous
| 2. appelle des OUTILS de lecture | <- données de CET utilisateur
| 3. rédige, cite, ou passe la main |
+-----+
|
v
Réponse écrite en base (from_user = false)

```

3.1 Le volet statique — la documentation

Découper les articles en fragments, calculer une empreinte par fragment, les stocker, et retrouver les plus proches de la question. Le volume attendu — quelques centaines de fragments — reste dans ce que le projet sait déjà faire en mémoire.

Une règle non négociable : l'agent ne répond à une question documentaire **que** si un fragment pertinent a été trouvé. Sinon il dit qu'il ne sait pas. Un assistant qui comble les trous est pire que pas d'assistant : il se trompe avec assurance, sur un sujet où l'utilisateur ne peut pas vérifier.

3.2 Le volet dynamique — les outils

C'est ce qui distingue un agent d'un robot documentaire. Quelques outils suffisent au départ, chacun couvrant une famille de questions réelles :

Outil	Répond à
etatDuProfil	« pourquoi je n'apparais pas dans les recherches ? »
mesCandidatures	« où en est ma candidature ? »
monAvancementJeux	« pourquoi mon score est-il incomplet ? »
mesResultatsDeTests	« ai-je réussi le test ? »
pourquoiPasDeFitScore	« pourquoi cette offre n'a pas de note ? »

Le dernier mérite qu'on s'y arrête : « pourquoi mon score est vide » a **quatre** causes possibles dans le système actuel — offre non reliée à un métier, métier pas encore approuvé, aucun jeu joué, ou calcul en attente. Un outil qui tranche factuellement évite quatre allers-retours avec le support humain.

3.3 La règle de sécurité qui commande tout

L'identité de l'utilisateur vient du jeton, jamais des arguments fournis par le modèle.

Un outil qui accepterait un identifiant en paramètre serait exploitable : il suffirait d'écrire dans le chat « affiche les candidatures de l'utilisateur X » pour que le modèle appelle l'outil avec l'identifiant d'un autre. Chaque outil doit donc dériver l'identité du Principal de la requête, et ignorer tout ce que le modèle propose sur ce point.

C'est le point d'ingénierie le plus important de ce plan. Tout le reste se rattrape ; une fuite de données entre comptes, non.

3.4 Pourquoi des outils plutôt qu'un contexte pré-chargé

L'alternative serait de rassembler l'état de l'utilisateur dans le prompt à chaque message. Plus simple, mais deux défauts : on envoie au modèle des données dont il n'a pas besoin neuf fois sur dix — un principe de minimisation malmené — et le prompt grossit sans fin à mesure que le produit s'enrichit.

Les outils coûtent un aller-retour de plus, sur un chemin qui n'est pas critique : personne n'attend une réponse de support en 200 ms.

4. Ce qu'il faut écrire — le corpus

C'est le chemin critique. L'application compte 17 modules fonctionnels ; tous ne justifient pas un article, mais les suivants oui :

Domaine	Exemples d'articles
Fit Score	Comment il est calculé · Pourquoi il est vide · Pourquoi il a changé
Jeux	À quoi ils servent · Que se passe-t-il si j'en saute un · Puis-je rejouer
Candidatures	Les statuts · Qui voit quoi · Répondre à une présélection
Tests techniques	Comment ils comptent · Que vaut un test passé ailleurs
Profil	Complétude · CV · Ce qui est visible du recruteur
Recruteur	Créer une offre · Le métier obligatoire · L'alerte test manquant
Compte	Connexion, mot de passe, notifications, données personnelles

Un point de méthode : ces articles ne peuvent pas être obtenus en reformulant les documents internes. Le document de référence du Fit Score explique *comment le système fonctionne*, y compris ses défauts et ses décisions provisoires. Un article d'aide répond à *ce que l'utilisateur doit faire*. Ce sont deux textes différents.

Estimation : 25 à 40 articles courts. C'est le poste le plus lourd du projet, et il ne se code pas.

5. Les garde-fous

Règle	Pourquoi
Ne jamais affirmer un fait de compte sans résultat d'outil	C'est la frontière entre « j'ai vérifié votre compte » et une phrase inventée qui y ressemble.
Dire « je ne sais pas » et proposer l'escalade	Un aveu d'ignorance est réparable ; une réponse fausse ne l'est pas.
Ne jamais promettre une action	L'agent lit, il ne modifie rien. Pas de « j'ai réinitialisé votre mot de passe ».
Annoncer que l'interlocuteur est automatique	Exigence d'information, et cela conditionne la confiance dans la réponse.
Journaliser la question, les fragments cités et les outils appelés	Sans cela, une réponse contestée n'est pas explicable — même exigence que sur le Fit Score.
Adapter le ton au public	ResumeAudience existe déjà : un recruteur et un candidat ne posent pas les mêmes questions.

6. Le plan, étape par étape

Étape 1 — Rendre le centre d'aide fonctionnel sans agent [OK] faite le 20 août 2026

- [X] POST /help-chats — ouvrir une conversation
- [X] sendMessage au datasource mobile et **un bouton d'envoi** dans la barre
- [X] Migration **V64** : note, commentaire et date sur help_chats
- [X] POST /help-chats/{id}/rating — _selectRating et onSubmit branchés

- [x] Tests : 8 tests unitaires + vérification bout en bout sur API réelle (docs/help-center/verif_help_center.py)

541 tests, 0 échec · flutter analyze **0 erreur** · **schéma Flyway 64** · **20 vérifications vertes sur l'API réelle**, dont l'isolation entre utilisateurs.

Quatre défauts trouvés en chemin, qu'aucun test ne couvrait :

Défaut	Conséquence
HelpChatModel lisait un champ time que le serveur n'envoie pas	La liste plantait dès la première conversation réelle — invisible tant que la table était vide
Le message parsait la date comme un nombre	Le serveur émet de l'ISO (2026-08-20T14:48:55Z) : toDouble() sur une chaîne
Le dialogue de notation émettait le libellé traduit	En français il aurait envoyé « Médiocre » — refusé par le serveur
Le routeur fabriquait une conversation id : '1'	Identifiant inexistant : tout appel serveur échouait

Étape 2 — Écrire le corpus [OK] faite le 20 août 2026

- [x] **69 articles** rédigés — 5 900 mots, 18 catégories
- [x] Stockés en base (engagement.help_articles, migration **V65**), avec empreinte et date de mise à jour
- [x] Écrits depuis le code, **jamais** depuis les documents internes

Répartition : 30 articles candidat · 25 communs · 14 recruteur, sur 18 catégories.

Une première version s'était arrêtée à 35 articles, dans la fourchette annoncée plus haut. Cette fourchette avait été estimée **avant** de connaître l'étendue réelle du produit : 17 modules mobiles et 29 contrôleurs. La moitié des fonctionnalités n'était pas couverte — inscription, recherche, messagerie, appels, publications, vérification d'identité, résumé de compétences, suivi de progression, gestion des tests côté recruteur. Le corpus a été doublé.

Le choix de conception : les articles vivent dans des fichiers de ressources (resources/help/fr/*.md) et sont projetés en base au démarrage. Un texte se relit et se corrige en revue, ce qu'une longue suite d'INSERT rendrait pénible. La table existe parce que l'étape 3 doit attacher une empreinte numérique à chaque fragment, et ne la recalculer que lorsque le texte change — l'empreinte du contenu sert exactement à cela.

La synchronisation gère **trois** cas, et le troisième est le plus facile à oublier : un article *retiré des fichiers* est supprimé de la base. Sans cela, un article écarté parce qu'il était faux continuerait d'être servi par l'agent.

Deux garde-fous inhabituels, parce que la matière première est du texte :

- un test refuse tout **vocabulaire interne** dans le corpus — 18 termes surveillés, dont ceux qui trahiraient l'architecture ou les défauts connus ;
- un autre refuse toute **fonctionnalité inexistante**, à commencer par le « Wallet » de la maquette d'origine.

Une correction de fond au passage : la note de passage des tests techniques est de **70 %**, pas 60 %, et elle n'est plus réglable par offre. Les articles disent 70.

555 tests, 0 échec · **schéma Flyway 65** · **corpus chargé et vérifié au démarrage**.

[!] **Le corpus est en français uniquement**. Le schéma porte une colonne locale et l'anglais s'ajoutera sans migration, mais doubler le corpus double le coût de rédaction : c'est l'arbitrage n° 5 restant.

Étape 3 — Le volet documentaire [OK] faite le 21 août 2026

- [x] Clé d'embeddings : elle **était configurée**, mais dans le .env de la racine alors que Maven tourne depuis backend/ — d'où un NoOpEmbeddingPort silencieux
- [x] **114 fragments** découpés par paragraphes, empreintes calculées et stockées (V66)
- [x] Recherche en mémoire, sur le modèle d'EmbeddingCodec
- [x] L'agent répond **en citant l'article**, ou avoue son ignorance

- [x] Vérifié en conditions réelles : docs/help-center/verif_agent.py, tout vert

572 tests, 0 échec · schéma Flyway 66 · 114 empreintes réutilisées au redémarrage, aucun appel réseau inutile.

Le seuil sémantique ne peut pas trancher seul — mesuré

	Score
Vraie correspondance	0,92
Meilleur résultat d'une question sans rapport	0,80

Six centièmes d'écart. Les préfixes « query: » / « passage: » recommandés pour ce modèle ne l'élargissent pas. Un seuil seul laissait donc passer « quelle est la recette de la tarte aux pommes ? » à **0,91** contre l'article sur le mot de passe oublié.

La correction : le sens **classe**, les mots **gardent la porte**. Un fragment doit partager un quart des mots de la question pour être éligible ; parmi les éligibles, la proximité sémantique ordonne. Une reformulation garde ses chances, une question étrangère au produit ne partage rien.

Quatre défauts trouvés en chemin

Défaut	Portée
Le modèle Groq configuré n'existe plus (404 model_not_found)	Cassait aussi la génération de tests et les résumés IA, silencieusement
Une réponse non-2xx du service d'empreintes repartait sans aucun journal	Clé invalide, quota ou modèle froid étaient indiscernables d'un service non configuré
« quelle » comptait comme un mot porteur de sens	Suffisait à faire passer une question hors sujet — c'est ce qui a fait échouer le test
Deux violations de couches (EmbeddingCodec, puis la recherche)	Rattrapées par ArchitectureTest, corrigées par un port

Étape 4 — Le volet dynamique (2 à 3 jours)

- [] Les cinq outils de lecture
- [] **Identité dérivée du jeton** — et un test qui tente explicitement de lire le compte d'un autre utilisateur
- [] Le « is typing » devient réel : il dure le temps de la génération
- [] Génération en écoute d'événement, pour que l'envoi du message ne fasse pas attendre

Étape 5 — L'escalade vers un humain (à décider)

C'est le même problème que la modération de la détection de fraude : **aucune interface d'administration n'existe dans ce projet**. L'approbation des métiers se fait déjà à la main. Trois besoins réclament désormais le même back-office manquant.

7. Estimation

Étape	Durée	Nature
1 — Centre d'aide fonctionnel	1 j	Code
2 — Corpus	3 à 5 j	Rédaction
3 — Volet documentaire	2 j	Code
4 — Volet dynamique	2 à 3 j	Code

Étape	Durée	Nature
5 — Escalade humaine	—	Décision

Environ 8 à 11 jours, dont près de la moitié sans écrire une ligne de code.

8. Ce qui reste à trancher

#	Point
1	L'escalade — qui répond quand l'agent ne sait pas, et depuis quelle interface ?
2	La clé d'embeddings — à configurer, sinon le volet documentaire ne peut pas exister
3	Le périmètre des outils — jusqu'où l'agent lit-il le compte ? Les résultats de tests en font-ils partie ?
4	La conservation — combien de temps garde-t-on les conversations d'aide ?
5	Le multilingue — l'application est bilingue ; le corpus doit-il l'être dès le départ ?

Module Engagement — projet Zennyt. Plan établi le 15 août 2026 sur l'état réel du dépôt à cette date.